

← Back to exams

Skill exam, week 5

140 min · 195 marks · pass 78

Opens Fri, 11 Sept, 9:00 am IST Last date to finish Mon, 14 Sept, 11:59 pm IST

YOUR SCORE

130/195 67%

Passed

Pass mark 78 (40%) · 52 above the line · finished in 2h 0m · submitted Mon, 14 Sept, 12:42 pm IST

Proctoring: CLEAN · 4 flags

YOUR SKILL SCORECARD

Strongest Problem solver · Work on Coder

Coder	15/40 · 38%
Can you write correct, clean code under time pressure? Real functions, real edge cases, real bugs to find.	
Developer	24/28 · 86%
Can you work inside a real codebase and team? APIs, data, Git, testing, debugging, and shipping without breaking things.	
Architect	18/30 · 60%
Can you design a system and defend the trade-offs? Scale, cost, failure modes, and what you would NOT build.	
AI engineer	26/41 · 63%
Can you build with LLMs the way production teams do? Prompts, RAG, agents, evals, cost and failure handling.	



Entrepreneur

22/24 · 92%

Can you turn work into value? Scoping, prioritising, saying no, pricing, and knowing when something is not worth building.

Communicator

13/20 · 65%

Can you explain your work so a manager, a client and a new teammate all understand it? Clear, short, honest.

Problem solver

12/12 · 100%

Can you think when there is no template? Logic, estimation, data reasoning and spotting the flaw in an argument.

 **Certificate issued**
EUEX-G3EIALBZGR

[View / download certificate](#)

FEEDBACK

You passed the exam with a score of 130/195, demonstrating strong performance in the Developer, Entrepreneur, and Problem Solver sections. However, there are significant areas for improvement, especially in the Coder section where you scored the lowest.

STRENGTHS

- Strong performance in Developer section with 86%
- Excellent score of 92% in Entrepreneur section
- Perfect score in Problem Solver section

AREAS OF IMPROVEMENT

- Focus on improving coding skills, especially in writing complete functions
- Practice more coding questions to enhance problem-solving speed and accuracy

TOPIC MASTERY

Coder · weak

Developer · strong

Architect · developing

AI engineer · developing



Entrepreneur · strong

Communicator · developing

Problem solver · strong

NEXT STEPS

Dedicate time to practice coding exercises, especially focusing on function writing and logic implementation.

Review the feedback on your coding questions and try to understand the common pitfalls that led to the low score.

Per-question breakdown

QUESTION 1 · OBJECTIVE

2/2 marks

What exactly does this Python code print?

```
cart = [{"pen", 10}, {"bag", 50}, {"pen", 5}]
```

```
totals = {}
```

```
for item in cart:
```

```
    name, qty = item
```

```
    totals[name] = totals.get(name, 0) + qty
```

```
    qty = 0
```

```
print(cart[0][1], totals["pen"])
```

YOUR ANSWER

10 15

MODEL ANSWER



10 15

QUESTION 2 · MCQ

2/2 marks

You are reviewing a Week 5 Python package named `calculator_tools`. The file structure is:

```
calculator_tools/  
  __init__.py  
  operations.py  
  main.py
```

`operations.py` contains:

```
def add(a, b):  
    return a + b
```

`main.py` contains:

```
from calculator_tools.operations import add
```

```
print(add("10", 5))
```

What is the best first review comment?

Change `add` to print the result instead of returning it, because CLI users need visible output.



Convert both inputs to strings inside add so the expression always works.

The caller should validate or convert input types before calling add, or add should clearly document and enforce a numeric contract; currently this will raise a `TypeError`. CORRECT

Move operations.py code into `__init__.py` so imports become simpler and the `TypeError` disappears.

QUESTION 3 · SUBJECTIVE

7/10 marks

You have to architect a Python CLI "Student Result Processor" for a coaching centre using only topics covered so far: Python basics, lists/dictionaries/sets/tuples, loops, functions, exception handling, logging, packages/modules, and local files. Constraints: 12,000 students per monthly exam; input is one or more CSV-like text files totalling up to 80 MB; runs on an office laptop with 2 CPU cores and 4 GB RAM; must finish in under 6 minutes for a normal run; 2 beginner Python developers will maintain it; no database, web server, or cloud service is approved this month; budget is zero. The current prototype loads the whole file into nested lists, crashes on one bad row, and overwrites the previous output file. Write a design proposal. You must explicitly state the likely bottleneck, your concrete component/module choices and why, at least 4 failure modes with handling, the main trade-offs, what you would defer, and a rollback or migration plan from the prototype.

YOUR ANSWER

Use a modular CLI with reader, validator, processor, writer and logger modules. main bottleneck is memory and I/O from 80MB files so process line by line instead of loading everything.

handle: bad rows -> log/skip, missing files -> clear error, invalid marks -> reject/log, existing/output failure -> time stamp

trade-off : streaming is slightly more complex but memory efficient and maintainable. Defer DB, cloud, web api and multiprocessing as unnecessary.



Migrate gradually compare new vs prototype outputs keep. prototype as rollback until verified

MODEL ANSWER

The dominant bottleneck is not CPU; it is file I/O, memory use, and row parsing. With up to 80 MB of text input and only 4 GB RAM, loading everything into nested lists is risky and unnecessary. Since the run must finish in under 6 minutes on a 2-core office laptop, the safest design is a streaming, line-by-line processor that does one pass over the files, keeps only small in-memory summaries, and writes output incrementally. This also suits two beginner maintainers because it is simpler to reason about than a large in-memory design, and it respects the zero-budget constraint by using only the standard library and local files.

I would structure it as a small package with clear modules: `reader.py` to open files and yield rows one at a time; `validator.py` to check row length, student id format, and mark fields; `calculator.py` for pure functions that compute totals, averages, grades, pass/fail, and ranking rules; `reporter.py` to write the final CSV/text report and quarantine file; `logger_setup.py` or `config.py` for logging format and paths; and `main.py` as the CLI entry point. The key rule is that calculation functions must be pure and separate from file handling, so they are easy to test and maintain. Standard file handling with `open(...)` and the logging module is enough and gives traceable audit logs without extra dependencies.

Failure handling should be explicit. If an input file is missing, log it as an error and stop that file while continuing with others if any remain. If a row is malformed or has missing columns, skip it, log the line number, and write it to a quarantine file for review. If marks are non-numeric or out of range, reject that row and count it in the summary. If a duplicate student id appears, either keep the first valid record and log the duplicate, or mark it as a conflict based on a fixed rule. If output writing fails because of permission error, disk full, or partial write, write to a temporary file first and only rename it to the final name on success; otherwise preserve the previous output and log the failure. The final log should include counts for processed rows, accepted rows, skipped rows, duplicates, and output errors.

The main trade-off is between simplicity and flexibility. A database would make searching and updating easier, but it is not approved and adds complexity. Streaming is slightly less convenient than loading the whole file, but it is much safer for memory and faster for large files. Strict validation improves data quality, but I would still allow partial success so one bad row does not stop the whole exam. I would defer a GUI, cloud sync, real-time dashboard, multi-user editing, and advanced



analytics because they are not needed for a monthly batch process and would slow delivery.

For migration, I would keep the old prototype as a fallback, introduce versioned output names such as `results_2026_09_v1.csv`, and run the new processor in dry-run mode on a sample file to compare counts and totals against the old script. After that, process one real exam in parallel and compare outputs before switching fully. If the new version fails, we can immediately revert to the old script and keep the last known good output because the new writer always uses a temporary file and preserves the previous report until success.

HOW MARKS WERE AWARDED

10 marks: Bottleneck and constraints, 2 marks: identifies the dominant bottleneck as file I/O plus memory/row parsing rather than CPU, and ties decisions to 80 MB, 4 GB RAM, 6 minute limit, 2 beginner maintainers, and zero budget. Concrete architecture, 3 marks: proposes a clear package/module structure such as reader, validator, calculator, reporter, logger/config; uses streaming line-by-line processing or chunking instead of loading everything; separates pure functions for marks calculation from I/O; uses standard Python files and logging with specific reasons. Failure handling and observability, 2 marks: covers at least 4 specific failures such as missing file, malformed row, non-numeric marks, duplicate student id, partial output write, disk full, permission error; explains continue/skip/quarantine rules and logging summary counts. Trade-offs and deferrals, 2 marks: explains trade-offs such as simple CSV/text files versus database, streaming versus easier whole-file code, strict validation versus accepting partial results, performance versus maintainability; clearly names features to defer such as GUI, cloud sync, real-time dashboard, multi-user editing. Migration or rollback plan, 1 mark: includes a safe migration path from prototype, versioned output names, backup of old output, dry-run or sample comparison, and a way to return to the old script if the new one fails. Penalise generic buzzwords like scalable, microservices, AI, or cloud without connection to the given constraints.

The student identifies the bottleneck and proposes a modular structure, but lacks detail on specific failure handling and concrete reasons for design choices.

QUESTION 4 · MCQ

2/2 marks



You are adding an AI step to a Python student result processor. The model receives a student dictionary and must return a machine-readable result, not a paragraph. Which prompt is best for reducing ambiguous output and hallucinated fields?

Read this Python dictionary: {student}. Return exactly one JSON object with only these keys: name, total, status, reason. status must be PASS if total >= 120, otherwise FAIL. If any mark is missing or non-numeric, return exactly {"error": "invalid_marks"}. Do not add extra text. **CORRECT**

You are a helpful teacher. Read {student}, calculate the result, and explain clearly whether the student passed. Add any useful advice.

Think step by step about {student}. Be creative and include all information that may help the school understand the student.

Read {student}. Return the result in any format that looks good to the user, but make sure it is correct.

QUESTION 5 · MCQ

2/2 marks

You have 2 evenings (about 6 hours total) to build a Python CLI MVP for a hostel mess manager. There are 80 students. The manager's main complaint is: "Every night I spend 45 minutes copying meal attendance from WhatsApp messages into a notebook and I make mistakes." You can build only one feature now. Which feature is the best MVP choice?

A CLI that lets the manager enter each student's present/absent status, saves a daily CSV/text file, and prints the total meals for the day. **CORRECT**

A full menu planning system with weekly nutrition scores, student ratings, and colour-coded reports.



A login system with admin and student roles before any attendance feature is built.

A Python package with reusable attendance classes, but no working CLI yet.

QUESTION 6 · SUBJECTIVE

8/10 marks

You built a Python Student Result Processor that reads student marks, calculates pass/fail, and writes a result. A non-technical programme coordinator asks why you spent time adding exception handling and an activity log instead of only writing the calculation code. Write your reply in under 120 words. Your reply must explain the technical decision, the benefit to students or staff, and one remaining risk or limitation.

YOUR ANSWER

I added exceptions handling so one invalid record does not crash the whole program. the activity log records successful and error action, which makes finding the issue easier.
this gives students more reliable results and helps staff troubleshoot quickly.
A remaining risk is that unexpected input formats may still require manual correction.

MODEL ANSWER

I added exception handling so the program does not crash if marks are missing, typed wrongly, or out of range. I added an activity log so staff can see what happened in each run and quickly trace any problems. This makes the system more reliable for students and saves staff time when checking results or fixing mistakes. The main benefit is that fewer results are lost and errors can be found faster. One limitation is that logging and error handling cannot fix bad data by themselves; if the source file is wrong, someone still has to correct it at the source.

HOW MARKS WERE AWARDED

Clarity for a non-technical stakeholder: 3 marks for plain language, clear sentences, and avoiding unexplained Python jargon. Technical decision and reasoning: 3 marks for accurately explaining exception handling and logging as ways to



handle errors and trace what happened. Benefit and risk: 2 marks for naming a concrete benefit and honestly stating one realistic remaining risk or limitation. Brevity and structure: 2 marks for staying under 120 words, removing padding, and giving a focused answer.

The response is clear and concise, effectively explains the technical decisions and their benefits, but could improve on sentence structure and minor grammatical issues.

QUESTION 7 · OBJECTIVE

2/2 marks

A cohort has 30 students. Each student runs a Python CLI tool 5 times per week for 24 weeks. Each run writes 20 log lines, and each log line is estimated as 150 bytes. Using 1 MB = 1,000,000 bytes, what is the total log size for the whole cohort? Give the answer in MB.

YOUR ANSWER

10.8 MB approx

MODEL ANSWER

10.8 MB

QUESTION 8 · OBJECTIVE

2/2 marks

What exactly does this Python code print?

```
def safe_int(x):  
    try:  
        return int(x)  
    except ValueError:
```



```
return -1  
  
finally:  
    print("checked", x)
```

```
print(safe_int("08") + safe_int("x"))
```

YOUR ANSWER

```
checked 08  
checked x  
7
```

MODEL ANSWER

```
checked 08  
checked x  
7
```

QUESTION 9 · MCQ**2/2 marks**

A teammate wrote this error handling in a student result processor:

```
try:  
    percentage = total / max_marks * 100  
except Exception:  
    print("Something went wrong")  
    percentage = 0
```

Which replacement is the best engineering practice based on exception handling and logging?



Keep catching Exception, but change the message to print("Error in percentage calculation") so users know the failing area.

Catch the likely specific errors, log useful context such as student id and values, and either skip **CORRECT** that record or return a clear failure result without hiding the bug.

Remove try/except completely because Python errors should always crash the program.

Wrap the whole application in one try/except at the top and always continue with default values.

QUESTION 10 · SCENARIO

5/10 marks

Scenario: Your team built a Python CLI "File Organizer" for a training lab. It scans a shared folder and moves files into subfolders by extension and date. New constraints: 35 students submit files during a 20-minute lab window; around 25,000 files are created per day; average file size is 200 KB but some are 500 MB videos; the lab PC running the tool has 8 GB RAM and a slow external HDD; target is that a run should complete within 10 minutes and never permanently lose a student file; only 1 maintainer is available; budget is zero. The current design first builds a huge list of every file path, then moves files directly, and if interrupted some files disappear from the expected place. Recommend an architecture and operating plan. Include the bottleneck, concrete Python-level choices, failure modes, trade-offs, what to defer, and rollback/migration plan.

YOUR ANSWER

Bottleneck is slow HDD I/O, especially large videos. stream directory entries instead of building huge list and process files one by one use modules to scan classify move and logging and before moving, verify destination handle duplicate and log the source and



destination, copy very and delete when crossing files system, handle permission errors missing files duplicate name disk -full and interruptions.

MODEL ANSWER

The main bottleneck is not Python CPU time; it is slow disk I/O, especially moving large files on an external HDD. With 25,000 files per day and some 500 MB videos, a design that builds one huge list and then does direct moves will spend most time waiting on the disk and also risks data loss if the process is interrupted halfway through.

I would change the program into four small modules: scanner, planner, mover, and logger. The scanner should walk the tree incrementally with `os.scandir()` or `os.walk()`, yielding one file at a time instead of storing everything in memory. The planner should write a manifest first, for example CSV or JSON lines with source path, target path, size, mtime, and planned action. Only after the plan exists should the mover start changing files. All actions should use the standard logging module so every move, skip, error, and recovery step is recorded.

For safety, use copy-then-verify-then-delete when the source and target are on different disks. If a file is on the same filesystem and the destination path is safe, `os.replace()` is faster and effectively atomic. Before moving, skip files that look still being written by checking that size and modification time are unchanged across two scans a short interval apart. For each file, create the destination path, copy or rename it, verify size or checksum for risky large files, then delete the source only after verification.

Failure modes to handle include interruption during a move, duplicate filenames, permission denied, disk full, very large video files, files changing during processing, invalid paths, and external HDD disconnects. Recovery should read the manifest and log on restart, detect completed items, retry incomplete ones, and leave any uncertain file in its original place or a quarantine folder rather than losing it. If a filename collides, append a timestamp or numbered suffix. If disk space is low, pause safely and report it.

Trade-offs: copy-verify-delete is slower than direct move, but it is safer and meets the no-data-loss requirement. A manifest adds complexity, but it gives recoverability and rollback. I would use scheduled batches during the 20-minute lab window rather than real-time watching, because it is simpler and more reliable on one weak PC. I would defer a GUI, database, cloud backup, antivirus integration, and multi-machine coordination because there is no budget and they are



not needed for this lab.

For rollout, I would add a dry-run mode first, test on a small folder, keep the old script available in parallel, and only then enable real moves. The original folder should remain untouched until each file is verified, and the manifest should support undo if a release must be rolled back.

HOW MARKS WERE AWARDED

10 marks: Bottleneck and risk diagnosis, 2 marks: names slow disk I/O and large file moves, not Python computation, as the bottleneck; identifies data-loss risk from direct moves and interruptions. Concrete design choices, 3 marks: proposes walking files incrementally rather than building one huge list, using functions/modules such as scanner, planner, mover, logger; writes a move plan or manifest before changing files; uses copy-then-verify-then-delete or same-disk rename where safe; skips files still being written using size/time checks; uses standard logging. Failure modes, 2 marks: handles at least 4 specific cases such as interruption during move, duplicate filenames, permission denied, disk full, very large video file, file changed during move, invalid path, or external HDD disconnect; gives concrete recovery behaviour. Trade-offs and deferrals, 2 marks: explains trade-offs such as copy safety versus speed, manifest complexity versus recoverability, running scheduled batches versus real-time watching, simple extension rules versus content inspection; defers non-covered or nonessential features such as GUI, cloud backup, database, automatic antivirus scanning, or multi-machine coordination. Rollback/migration, 1 mark: includes a reversible rollout such as dry-run mode, processing a small folder first, keeping original folder untouched until verification, manifest-based undo, and retaining the old script temporarily. Penalise answers that only list tools or say use cloud/database without respecting zero budget and course coverage.

The answer identifies the bottleneck and some design choices but lacks clarity on failure modes, trade-offs, and rollback plans.

QUESTION 11 · MCQ

2/2 marks

A course-support chatbot will answer from a 60-page FAQ. The bot first retrieves text chunks and then sends the best chunks to the model. The FAQ contains headings, policy tables, dates, and code examples.



Which chunking strategy is most likely to improve answer accuracy and citations?

Store the whole FAQ as one chunk so the model always has full context.

Split every 200 characters, even if it cuts a table row or code block, because smaller chunks are always better.

Split mainly by headings and subheadings, keep tables/code blocks intact, attach metadata such as page, heading, and version, and allow a small overlap between neighboring chunks. **CORRECT**

Shuffle sentences before chunking so each chunk has a mixture of topics and the retriever has more variety.

QUESTION 12 · MCQ

2/2 marks

A local tuition teacher asks you to build a Python CLI fee reminder tool. She has 12 students, spends about 10 minutes per week checking unpaid fees, and has a budget of ₹500. You estimate even a simple tool will take 8 hours. Your normal minimum value for 8 hours is ₹4,000. What is the best entrepreneurial decision?

Build the custom CLI anyway because every automation idea should become software.

Do not build a custom tool now; suggest a simple spreadsheet/WhatsApp checklist and revisit only if students or time spent increases. **CORRECT**

Build a complex app with login, database, and dashboards so the teacher sees more value.



Accept ₹500, but reduce quality by skipping error handling and testing.

QUESTION 13 · SUBJECTIVE**5/10 marks**

You are opening a pull request for a Python CLI Productivity Tool built in Week 5. The change moves repeated code into functions, separates helper code into a small module, adds basic exception handling for invalid user input, and logs completed actions to a file. You also promised to submit today, but testing found that the log file path fails on one teammate's laptop, so delivery will be delayed by one day. Write two items: 1. a pull request description for reviewers; 2. a short status update to your mentor admitting the delay and proposing a fix. Keep the total answer under 180 words. Do not claim tests passed unless you mention what was actually tested.

YOUR ANSWER

PR description: Refactored repeated code into functions and move helpers code into functions. added invalid input handling and activity logging. Basic CLI actions can be tested but testing found the log path fails on one teammate laptop.

status: I will miss todays delievery by one day. I found a platform specific log path issue during testing. i will replace the fixed path with a portable path, retest on both laptop and submit it by tomorrow

MODEL ANSWER

1. Pull request description: This PR refactors the Week 5 Python CLI tool by moving repeated logic into reusable functions, splitting helper code into a small module, adding basic exception handling for invalid user input, and logging completed actions to a file. These changes improve readability, reduce duplication, and make the CLI easier to maintain. I tested the main command flow and invalid-input handling locally. Risk to review: the log file path is not portable on one teammate's laptop, so please check path handling carefully.

2. Status update to mentor: I need to delay delivery by one day. Testing found that the current log file path fails on one



teammate's laptop, so I should not submit today. I'll fix this by switching to a more portable, configurable path and re-test on both machines, then submit tomorrow.

HOW MARKS WERE AWARDED

Pull request description: 3 marks for a clear summary, specific changes, and reviewer-relevant context without unnecessary background. Testing and risk honesty: 2 marks for stating what was tested or not tested and identifying the log file path risk accurately. Status update: 3 marks for admitting the delay directly, explaining the cause, giving a one-day recovery plan, and naming the proposed fix. Audience, tone, and brevity: 2 marks for professional tone, suitable structure for PR plus mentor update, minimal jargon, and staying under 180 words.

The pull request description lacks clarity and specific details, while the status update admits the delay but contains several grammatical errors and lacks a clear recovery plan.

QUESTION 14 · OBJECTIVE

2/2 marks

What is the exact output of this Python code?

```
nums = [1, 2, 4, 6, 7]
for x in nums:
    if x % 2 == 0:
        nums.remove(x)
print(nums)
```

YOUR ANSWER

[1, 4, 7]

MODEL ANSWER



[1, 4, 7]

QUESTION 15 · OBJECTIVE

2/2 marks

A package folder named `calculator_tools` contains these files:

`calculator_tools/__init__.py` is empty.

`calculator_tools/operations.py` contains:

```
def add(a, b):  
    return a + b
```

`app.py` contains:

```
1 import calculator_tools  
2 print(calculator_tools.add(2, 3))
```

When running `app.py`, which line in `app.py` contains the bug that causes the program to fail? Answer with only the line number.

YOUR ANSWER

2

MODEL ANSWER

2

QUESTION 16 · MCQ

2/2 marks



A function in an e-commerce dataset cleaner is intended to remove duplicate product IDs while preserving the first occurrence order. Which implementation best matches that contract?

```
def unique_ids(ids): return list(set(ids))
```

```
def unique_ids(ids): result = [] for product_id in ids: if product_id not in result:  
result.append(product_id) return result
```

CORRECT

```
def unique_ids(ids): ids.sort() return ids
```

```
def unique_ids(ids): for product_id in ids: if ids.count(product_id) > 1: ids.remove(product_id) return ids
```

QUESTION 17 · SCENARIO

6/10 marks

Scenario: A small sales team asks for a Python CLI "Activity Logger" similar to the course task. It records actions like login, call_made, order_created, and error_seen. Constraints: 18 users share one Windows network folder; peak is about 1,500 events per hour; each event must be recorded within 2 seconds from the user's point of view; monthly budget is zero; the team has 2 Python beginners; data must be usable for a weekly summary report; losing logs is worse than having duplicate logs. The prototype appends every event to one shared text file. During testing, lines are sometimes mixed together and the report script crashes on malformed lines. Design the next version using only local/network files, Python functions/modules, exception handling, and logging. Include bottleneck, component choices, failure modes, trade-offs, deferrals, and migration/rollback plan.

YOUR ANSWER

bottleneck is concurrent writes by 18 users to one shared file can cause mixed or corrupt lines, give each user and process its own append log file with timestamp or id, then merge



them for the weekly report. Use modules for event checking file writing and error log. Trade-off multiple files and merge logic but reliability improves. defer db/server/cloud. migrate by running new logs alongside the prototype, compare reports then switch, keep old files for rollback

MODEL ANSWER

The main bottleneck and risk is concurrent writes by 18 users to one shared network text file. On a Windows network folder, file locking and latency can interleave lines or delay writes, so the current design is unsafe. Because the business rule says losing logs is worse than having duplicates, I would optimise for log integrity and persistence first, even if that means some duplicate events must be tolerated and removed later in reporting.

I would change the design to per-user or per-machine append-only log files, ideally split by day, for example logs/2026-09-10/userA.log. Each event should be one small structured record, such as CSV or JSON-like text, containing timestamp, user, action, event_id, and optional details. The writer module should only format and append one record, then flush/close immediately. A separate parser/validator module should read records and reject or flag malformed ones. A reporter module should build the weekly summary from validated records only. A config module can hold the log path, file naming, and fallback location.

Failure handling should be local and practical. If the network folder is unavailable, the app should write to a local fallback queue file and retry later. If a write is interrupted, the next run should continue appending and the reporter should ignore or report partial lines. If a record is malformed, the parser should skip it, count it, and continue rather than crashing. If a duplicate event_id appears, the summary should deduplicate where possible but still keep both raw records. If the disk is full or there is a permission error, the app should show a clear error and preserve any unsent local queue. If timestamps are inconsistent across PCs, the report should sort primarily by record time and treat time as approximate.

The trade-off is that many small files are less tidy than one file, but they avoid write collisions and are easier to recover from. Structured text is stricter than free text, but it makes weekly reporting reliable. I would defer a database, message queue, dashboard, real-time analytics, and authentication system because they are outside the zero-budget, file-based scope.



For migration, I would freeze the old shared file, archive it, and start date-based per-user logs from a chosen cutover date. The report script should read both the old shared file and the new files during the transition. If the new approach causes problems, rollback is simple: switch the writer back to the old append method for one release while keeping the archived logs intact.

HOW MARKS WERE AWARDED

10 marks: Bottleneck and correctness judgement, 2 marks: identifies concurrent writes to one shared network file and network-folder latency as the main bottleneck/risk; states that log integrity matters more than perfect deduplication because losing logs is worse than duplicates. Concrete architecture, 3 marks: recommends per-user or per-machine append-only log files or daily files instead of one shared file; uses a structured line format such as delimited text or JSON-like records with timestamp, user, action, id; separates writer, validator/parser, reporter, and config/logger modules; keeps each write small and handles exceptions locally. Failure modes and recovery, 2 marks: covers at least 4 specific failures such as network folder unavailable, partial/malformed line, duplicate event, clock/time inconsistency, permission error, disk full, report encountering bad records, or app crash mid-write; gives practical handling such as local fallback queue, skip-to-bad-record file, retry, summary error counts, or idempotent event ids. Trade-offs and deferrals, 2 marks: explains trade-offs such as many small files versus one simple file, local fallback causing delayed central reports, structured logs being stricter but easier to parse, accepting duplicates versus risking loss; defers database, message queue, web dashboard, authentication system, and real-time analytics as outside current constraints. Migration/rollback, 1 mark: describes migrating from the shared file by freezing/archiving it, starting date-based per-user logs, making the reporter read both old and new formats during transition, and keeping a rollback path to the old append method for one release. Penalise generic claims like make it scalable or use microservices without naming the file-concurrency bottleneck and concrete file-based plan.

The answer identifies the main bottleneck and proposes a per-user log file approach, but lacks detail on failure modes, structured logging, and specific trade-offs.

QUESTION 18 · MCQ

2/2 marks



Your Python CLI assistant has two possible operations: `calculator(expression)`, which is cheap and deterministic, and `model(prompt)`, which is slower and costs money. The user asks: "What is 18% GST on 2499?" What is the best tool-use loop for low cost and low hallucination risk?

Send the question to the model first, ask it to reason step by step, then call `calculator` only if the model says it is unsure.

Detect that this is exact arithmetic, call `calculator` once with the parsed expression, log the tool **CORRECT** call, and return the result without using the model unless parsing fails.

Call the model three times and return the majority answer, because voting is safer than tools.

Ask the model to generate Python code, run the code automatically, and return whatever it prints.

QUESTION 19 · SCENARIO

9/10 marks

You are asked to build a Python CLI productivity tool for a small delivery shop in 3 days. The owner has 2 staff, 120 deliveries per week, and loses about ₹2,000 per week because parcels are forgotten or assigned twice. Budget is ₹6,000. You can spend about 12 hours. Possible features: A) add delivery with customer name, phone, area, due date; B) mark delivery as pending/done; C) list today's pending deliveries; D) save/load data from a text/CSV file; E) staff login; F) route optimisation; G) automatic SMS reminders; H) monthly profit dashboard; I) activity logging for errors/actions. Decide what to build for the MVP and what not to build. Include a one-line pitch, your reasoning using the numbers, and one success metric for the first week.

YOUR ANSWER



Build A,B,C,D and basic I (error log).
this directly prevents forgotten or duplicate delivery.
defer login, route optimization, SMS and dashboard .
with ₹2000 week losses a ₹6000 MVP can pay back in about 3 weeks

Pitch: simple cli to track every delivery and prevent missed or duplicate parcels

success: reduce delivery mistakes by atleast 50% in week 1

MODEL ANSWER

I would build a small Python CLI MVP focused only on preventing forgotten and double-assigned deliveries within the 12-hour limit.

My MVP would include A) add delivery with customer name, phone, area, and due date; B) mark delivery as pending or done; C) list today's pending deliveries; and D) save/load the records from a simple CSV or text file. I would also add I) basic activity/error logging if time permits, because it helps detect mistakes and gives a record of changes. I would not build E) staff login, F) route optimisation, G) automatic SMS reminders, or H) a monthly profit dashboard in the first version, because they are useful later but do not solve the immediate problem as directly as the core delivery tracking features.

The numbers justify this scope. The shop has 120 deliveries per week and is losing about ₹2,000 weekly due to parcels being forgotten or assigned twice. With only 2 staff, the biggest gain comes from one shared source of truth for delivery status, not from advanced features. The budget is ₹6,000, so a lightweight CLI in Python is reasonable and low-risk compared with paying for SMS, optimisation, or a full dashboard. In 12 hours, a simple menu-driven CLI with a CSV file is realistic and can start reducing losses immediately.

My 3-day plan would be: day 1, build the delivery data structure and add/list functions; day 2, add status updates and file save/load; day 3, test with sample deliveries, add logging, and fix edge cases. I would use a list of dictionaries or a small CSV file, since that is simple and suitable for current Python skills.

One-line pitch: "A simple delivery tracker for your shop that stops parcels from being forgotten or assigned twice, using one shared list for today's deliveries."



Success metric for week 1: track all 120 deliveries with fewer than 2 missed or duplicate assignments, or at least cut such errors by 50% compared with the current week. I would check this by comparing the logged deliveries and final completed list against the original weekly count.

HOW MARKS WERE AWARDED

Decision and MVP scope (3 marks): clearly chooses to build an MVP within 12 hours and includes the core features that directly prevent forgotten/double-assigned deliveries, such as A, B, C, D, and optionally I; excludes non-essential features like E, F, G, H for now. Numerical value reasoning (2 marks): uses the given numbers to compare cost/time/value, e.g. ₹2,000 weekly loss, ₹6,000 budget, 120 deliveries/week, 12-hour limit, and explains why a small CLI can be justified if it reduces losses quickly. Prioritisation under deadline (2 marks): sequences work realistically for 3 days, identifies must-have versus later features, and mentions a simple data structure/file approach suitable for current Python skills. One-line pitch (1.5 marks): gives a concise customer-facing pitch focused on the shop's pain, not a generic technical description. Success metric and validation (1.5 marks): proposes a measurable first-week metric, such as reducing missed/duplicate deliveries by at least 50% or tracking all 120 deliveries with fewer than 2 errors, and explains how it will be checked.

The student effectively identifies core MVP features and justifies the choice with numerical reasoning, but could improve on detailing the prioritization and data structure approach.

QUESTION 20 · OBJECTIVE

2/2 marks

What is the exact output of this Python code?

```
orders = [  
    {"student": "A", "items": ["pen", "book"]},  
    {"student": "B", "items": ["pen"]},  
    {"student": "A", "items": ["bag"]}  
]
```



```
summary = {}  
for order in orders:  
    summary[order["student"]] = len(order["items"])  
print(sum(summary.values()))
```

YOUR ANSWER

2

MODEL ANSWER

2

QUESTION 21 · MCQ

2/2 marks

Which function definition avoids accidentally sharing the same task list between different calls when no list is passed?

```
def add_task(task, tasks=[]): tasks.append(task) return tasks
```

```
def add_task(task, tasks=None): if tasks is None: tasks = [] tasks.append(task) return tasks
```

 CORRECT

```
def add_task(task, tasks=list): tasks.append(task) return tasks
```

```
def add_task(task, tasks=()): tasks.append(task) return tasks
```

QUESTION 22 · MCQ

2/2 marks



You open an unfamiliar Python CLI productivity tool. At the top of tasks.py you see:

```
TASKS = []
```

```
def add_task(title):  
    TASKS.append(title)  
    return TASKS
```

A review asks for code that is easier to test and safer to reuse. Which change is best?

Keep the global list, but rename TASKS to tasks because lowercase variables are easier to read.

Change add_task to accept the current task list as an argument and return a new updated list or clearly mutate the passed list, avoiding hidden global state. **CORRECT**

Put TASKS inside a while loop so it resets every time the menu is displayed.

Add print(TASKS) inside add_task so tests can check the terminal output.

QUESTION 23 · CODE

6/15 marks

Write a Python function named `call_model_with_retries(model_call, prompt, max_attempts=3, base_delay=0.25, logger=None, sleep_fn=None)`. `model_call` is a callable that accepts `prompt` and returns the model response. Requirements: try at most `max_attempts` total attempts; if `max_attempts` is less than 1, raise `ValueError` before calling `model_call`; retry only `TimeoutError` and `ConnectionError`; do not retry `ValueError`, `TypeError`, or other exceptions; before each retry, wait using exponential backoff delays



base_delay, then base_delay*2, then base_delay*4, and so on; if sleep_fn is None, use time.sleep; if logger is provided, call logger.warning for failures that will be retried and logger.error for final failures or non-retryable failures; do not log the prompt text or response text; return the successful response; if all attempts fail, raise the last exception.

YOUR ANSWER

```
import time

def call_model_with_retries(model_call, prompt, max_attempts=3, base_delay=0.25,
                             logger=None, sleep_fn=None):
    if max_attempts < 1:
        raise ValueError("Invalid attempts")
    sleep_fn = sleep_fn or time.sleep

    for i in range(max_attempts):
        try:
            return model_call(prompt)

        except Exception as e:
            if logger:
                logger.error(type(e).__name__)
            raise
```

MODEL ANSWER

```
import time

def call_model_with_retries(model_call, prompt, max_attempts=3, base_delay=0.25,
                             logger=None, sleep_fn=None):
    """Call model_call(prompt) with retries for TimeoutError and ConnectionError only."""
    if max_attempts < 1:
        raise ValueError("max_attempts must be at least 1")

    if sleep_fn is None:
        sleep_fn = time.sleep

    last_exc = None

    for attempt in range(1, max_attempts + 1):
```



```

try:
    return model_call(prompt)
except (TimeoutError, ConnectionError) as exc:
    last_exc = exc
    if attempt == max_attempts:
        if logger is not None:
            logger.error("Model call failed after all retry attempts")
        raise
    if logger is not None:
        logger.warning("Model call failed; retrying")
    sleep_fn(base_delay * (2 ** (attempt - 1)))
except (ValueError, TypeError):
    if logger is not None:
        logger.error("Non-retryable model call failure")
    raise
except Exception:
    if logger is not None:
        logger.error("Non-retryable model call failure")
    raise

raise last_exc

```

Edge cases handled: `max_attempts < 1` raises `ValueError` before any model call. The function returns immediately on the first success, and it stops after exactly `max_attempts` total calls with exponential backoff only between failed attempts. Only `TimeoutError` and `ConnectionError` are retried; all other exceptions are not retried and are re-raised immediately. Logging never includes the prompt or response text, and `sleep_fn` can be injected for testing.

HOW MARKS WERE AWARDED

Correct retry control and exception behavior: 5 marks for respecting `max_attempts` as total attempts, raising `ValueError` for `max_attempts < 1` before any call, retrying only `TimeoutError` and `ConnectionError`, not retrying non-retryable exceptions, and re-raising the last exception after exhaustion. Backoff and sleep injection: 3 marks for correct exponential delays starting at `base_delay`, sleeping only between attempts, using `time.sleep` by default, and supporting `sleep_fn` for testing. Logging and privacy: 3 marks for using `logger.warning` on retryable failures that will be retried, `logger.error` on final or non-retryable failures, and avoiding logging prompt or response content. Clean Python structure: 2 marks for a clear function with necessary imports, no external packages, no global mutable state, and readable variable names. Edge cases



and return value: 2 marks for returning the first successful response immediately, avoiding off-by-one extra calls, and handling success after one or more failures.

The function has some correct structure and attempts to handle retries, but it contains syntax errors, does not implement exponential backoff, and does not correctly handle logging or exception types.

QUESTION 24 · SCENARIO

9/10 marks

A college club asks you to build a Python CLI event registration system in 5 days. They expect 35 attendees for one workshop. Current method: a Google Form already collects name, phone, and department, and one volunteer spends 20 minutes exporting the sheet and counting totals. Budget is ₹1,500. You estimate a basic CLI with add/search/export and error logging will take 10 hours; adding QR check-in will take another 12 hours. The club says, "Build whatever you think is best." Make a clear build/no-build decision. If you choose not to build, say what you would recommend instead. If you choose to build, define the smallest MVP. Include cost-versus-value reasoning, risks, and a one-line pitch or recommendation.

YOUR ANSWER

I Do not build now.
only 35 student attendees and the existing Google Form already works
saving 20 mins does not justify 10-12 development hours and ₹1500 budget.
keep Google form and use simple formula for totals
OR check-in adds more cost and complexity with little value
Recommendation: Improve the existing process instead of building unnecessary software

MODEL ANSWER

I would not build a custom Python CLI for this event.

The numbers do not justify it. You have 35 attendees, and the current process only costs one volunteer about 20 minutes to export the Google Sheet and count totals. A basic CLI would take me about 10 hours, and QR check-in would add



another 12 hours. For a one-workshop, one-time need, that is a lot of effort for very small time savings. With a ₹1,500 budget, the ROI is poor because the software cost in developer time is far higher than the operational time being saved.

My recommendation is to improve the existing Google Form/Sheet instead of building new software. Use the current form, add required fields and a duplicate check on phone number, sort/filter the sheet, and prepare a simple checklist for registration and check-in. If needed, create a second tab for “checked in” status and use conditional formatting to flag missing or duplicate entries. This solves the immediate problem with almost no setup time.

The main risks of building are overengineering, maintenance burden, and data-entry mistakes from a custom CLI that only one volunteer may use. QR check-in is especially hard to justify here because it adds 12 more hours but does not materially improve value for just 35 people.

One-line recommendation: use the existing Google Form and a cleaned Google Sheet workflow now; do not spend the budget and time on a custom CLI for a single small workshop.

HOW MARKS WERE AWARDED

Clear decision (2 marks): makes one defensible decision, preferably not building a custom CLI for this one small event unless a very narrow paid MVP is justified; avoids hedging such as saying both options equally. Cost-versus-value

reasoning (3 marks): uses the numbers: 35 attendees, 20 minutes saved, ₹1,500 budget, 10 hours basic build, 12 extra hours QR check-in, and explains why custom software has low ROI compared with the existing Google Form workflow.

Alternative or MVP scope (2 marks): if not building, recommends a practical non-build solution such as improving the Google Sheet with filters, duplicate checks, and a simple checklist; if building, limits scope to only the smallest feature set and explicitly rejects QR check-in.

Risk awareness (1.5 marks): identifies concrete risks such as overbuilding, data entry errors, maintenance burden, one-time usage, or QR check-in consuming time without enough value.

One-line pitch/recommendation (1.5 marks): provides a concise line aimed at the club, e.g. a recommendation to use the existing form plus a cleaned sheet rather than paying for unnecessary software.



The decision not to build is clear and well-justified, with strong cost-versus-value reasoning and a practical alternative, but could benefit from slightly more detail on risks.

QUESTION 25 · MCQ

2/2 marks

A mini e-commerce dataset records product views, cart additions, and purchases:

Product A: views 1000, cart additions 100, purchases 50

Product B: views 200, cart additions 80, purchases 40

Product C: views 500, cart additions 150, purchases 30

Product D: views 50, cart additions 40, purchases 20

Which product has the highest purchase rate per view?

Product A, because it has the highest number of purchases

Product B, because 40 out of 80 cart additions became purchases

Product C, because it has the highest number of cart additions

Product D, because 20 purchases from 50 views is the highest purchase-per-view rate

CORRECT

QUESTION 26 · MCQ

2/2 marks

What does this Python code print?



```
n = 12
while n > 1:
    if n % 5 == 0:
        break
    n //= 2
else:
    print("clean")
```

```
print(n)
```

clean 1

CORRECT

1

clean 0

6

QUESTION 27 · SCENARIO

8/10 marks

You pull a teammate's branch for the File Organizer Using Packages and Modules task. Running python main.py from the project root fails with:

Traceback (most recent call last):

File "main.py", line 2, in <module>



```
from organizer.files import move_file
```

ModuleNotFoundError: No module named 'organizer'

The branch contains:

file_organizer/

main.py

organiser/

__init__.py

files.py

files.py defines move_file. You must leave a PR review comment explaining what you would check first and what fix you would request. What would you do first and why?

YOUR ANSWER

I would first reproduce the failure from the document project root and compare the failing import with the actual package structure.

1. package / import spelling mismatch
2. actual repo structure versus traceback/import
3. fix: make organizer/organiser consistent
4. fix system env issue if any
4. verify: rerun from project root and test the import

MODEL ANSWER

First I would reproduce the failure exactly from the documented project root by running `python main.py` in `file_organizer/`, and I would confirm the current working directory and the interpreter being used before changing anything. That tells me whether this is a real project issue or just a local run/setup problem.

The immediate problem is a name mismatch: `main.py` imports `organizer.files`, but the actual package folder is `organiser/`.



Because Python resolves imports by package name, organizer does not exist, so the import fails.

I would request one consistent package name and have the code match it everywhere. Either rename the folder to organizer/ and keep the import as from organizer.files import move_file, or change the import to from organiser.files import move_file. In either case, `__init__.py` should remain in the package directory so it is treated as a package.

I would not suggest fixing this by editing `sys.path`, copying files into another folder, or relying on a VS Code run configuration, because those are fragile and hide the real packaging issue.

After the rename/import fix, I would verify it by rerunning `python main.py` from the project root and, if possible, adding a small smoke test that imports `move_file` and calls it on a simple sample file path to confirm the package imports and function work correctly.

HOW MARKS WERE AWARDED

Award 10 marks: 2 marks for first reproducing the issue from the documented project root command and confirming the current working directory/interpreter rather than guessing; 2 marks for identifying the concrete mismatch between imported package name 'organizer' and actual folder name 'organiser'; 2 marks for requesting one consistent package name and updating imports or folder names accordingly, with `__init__.py` kept in the package; 2 marks for avoiding fragile fixes such as editing `sys.path`, copying files into random folders, or relying on VS Code-only run settings; 2 marks for suggesting a small verification step, such as rerunning `python main.py` and adding a simple import/function smoke test for `move_file`.

The answer correctly identifies the import issue and suggests a consistent naming fix, but it lacks clarity on avoiding fragile fixes and does not explicitly mention keeping `__init__.py`.

QUESTION 28 · SCENARIO

8/10 marks



You are building a Python CLI productivity tool for students. A new AI feature converts a messy note into tasks with title, due_date, and priority. In testing, the model sometimes invents due dates, rewrites names incorrectly, takes 8 seconds, and costs too much because every note is sent to the model. Design a production-safe first version using only ideas that can be implemented with Python functions, dictionaries/lists, exception handling, and logging. Explain your approach for prompt design, guardrails against invented data, cost and latency control, and observability.

YOUR ANSWER

Use a strict prompt requiring only title, due_date and priority and explicitly say never invent missing values, return null when it is unknown. Validate date format valid priority and preserve names from source text. Reject or ask for confirmation on invented data. avoid model call when simple python tules can parse the note and limit retires time outs to control cost and latency. log latency success failure validation errors retires and cost-related counts without logging sensitive note content

MODEL ANSWER

I would ship a conservative first version that treats the model as an optional parser, not as the source of truth.

For prompt design, I would use a strict extraction prompt that says: "Extract tasks from the note and return only JSON with this exact schema: {"tasks":

[{"title":string,"due_date":string|null,"priority":"low"|"medium"|"high"}],"needs_confirmation":boolean}. Use the user's original wording for title as much as possible. Do not rewrite names. If a due date is not explicitly present, output null. If priority is not explicit, use null or a default only if the prompt defines one. Do not guess or infer missing information." I would also require the model to return nothing except JSON.

For guardrails, I would validate the parsed output in Python with dictionaries and lists. I would check that the top-level keys and task fields exist, that priority is one of the allowed values, and that due_date is either null or a simple date format I can parse safely. Any invented or malformed date, changed name, or missing required structure would make the result invalid. I would keep the original user note unchanged in the saved record for audit. If any task is uncertain, invalid, or partially extracted, I would not auto-save it as final; I would mark it needs_confirmation=True and ask the user to



confirm before creating tasks.

For cost and latency, I would avoid calling the model when simple rules are enough, such as notes with one obvious task or obvious due date patterns. I would also limit input length, truncate very long notes, and use a cheaper or faster path for short notes, for example regex-based extraction first and model only if that fails. I would add a timeout around the model call and a small retry only for transient failures.

For observability, I would log a request_id, timestamp, model name, elapsed time, success or failure, validation errors, and approximate character or token counts. I would not log the full note content; only metadata and error types.

If the model fails, times out, or returns invalid output, the safe fallback is to store the original note as an unprocessed inbox item and tell the user it needs review later.

HOW MARKS WERE AWARDED

Prompt and output contract: 2 marks for specifying a strict task extraction prompt with an exact schema, allowed values, and an explicit rule to use null or unknown instead of inventing missing due dates or names. Guardrails and validation: 3 marks for validating the model output in Python, rejecting or flagging invalid dates/priorities, preserving original user text for audit, and requiring confirmation before saving uncertain tasks. Cost and latency control: 2 marks for avoiding model calls when simple rules are enough, limiting input size, using a cheaper or faster path for short notes, and applying timeouts or retries only where useful. Observability: 2 marks for logging request id, timing, model used, success/failure, validation errors, and approximate token or character counts without logging sensitive note content. Practical failure handling: 1 mark for giving a safe fallback such as saving the original note as an unprocessed inbox item when the model call fails or output is invalid.

The answer covers prompt design, validation, cost control, and observability well, but lacks clarity on specific implementation details and practical failure handling.



A mentor says: "All 12 students who submitted the Application Activity Logger task used try/except blocks. Therefore, using try/except blocks causes students to submit the task successfully."

What is the main flaw in this argument?

It assumes causation from an observed pattern and gives no comparison with students who did not submit or submitted late **CORRECT**

It is wrong because try/except blocks can never be used in a logger application

It is wrong because 12 students is always enough to prove a programming rule

It is wrong because every Python program must use logging before try/except can work

QUESTION 30 · CODE

5/15 marks

Write a complete Python function named `process_results(records)`.

Input: `records` is a list of dictionaries. Each dictionary should have:

- "name": student name as a string
- "marks": a list of marks

Return: a tuple (`results`, `errors`).

`results` must be a list of dictionaries, in the same order as valid input records. For each valid student include:

- "name": cleaned name with leading/trailing spaces removed



- "average": average mark rounded to 2 decimal places
- "status": "PASS" if every mark is at least 40 and the average is at least 50, otherwise "FAIL"

errors must be a list of strings, one for each invalid record, in input order. Error strings must be exactly:

- "Record <index>: invalid name" if name is missing, not a string, or becomes empty after strip()
- "Record <index>: invalid marks" if marks is missing, is not a non-empty list, or contains any non-numeric mark, or any mark outside 0 to 100 inclusive

If both name and marks are invalid, report only the name error. Index is the 0-based position in records.

Example:

Input:

```
[  
  {"name": "Asha ", "marks": [80, 75, 90]},  
  {"name": "", "marks": [50]},  
  {"name": "Ravi", "marks": [39, 70]},  
  {"name": "Mina", "marks": [100, "90"]} ]
```

Return:

```
(  
  [  
    {"name": "Asha", "average": 81.67, "status": "PASS"},  
    {"name": "Ravi", "average": 54.5, "status": "FAIL"}  
  ],
```



```
["Record 1: invalid name", "Record 3: invalid marks"]
)
```

YOUR ANSWER

```
def process_results(records):
    results = []
    errors = []

    for index, record in enumerate(records):
        name = record.get("name")

        if not isinstance(name, str) or not name.strip():
            errors.append(f"Record {index}: invalid name")
            continue
        cleaned_name = name.strip()

        marks = record.get("marks")
        if not isinstance(marks, list) or len(marks) == 0:
            errors.append(f"Record {index}: invalid marks ")
            continue
        valid_marks = True
        for mark in marks:
            if(isinstance(mark, bool) or not isinstance(mark, (int, float)) or mark<0 or
mark>100):
                valid_marks = False
                break
        if not valid_marks:
            errors.append(f"Record {index}: invalid marks")
            continue
        average = round(sum(marks)/len(marks), 2)
        if all(mark >= 40 for mark in marks) and average >=50
            status = "PASS"
        else:
            status = "FAIL"

        results.append({

    })
```

MODEL ANSWER



```
def process_results(records):
    results = []
    errors = []

    for index, record in enumerate(records):
        name = record.get("name") if isinstance(record, dict) else None
        marks = record.get("marks") if isinstance(record, dict) else None

        # Validate name first; if invalid, report only name error
        if not isinstance(name, str) or name.strip() == "":
            errors.append(f"Record {index}: invalid name")
            continue

        # Validate marks
        if not isinstance(marks, list) or len(marks) == 0:
            errors.append(f"Record {index}: invalid marks")
            continue

        valid_marks = True
        total = 0
        all_at_least_40 = True

        for m in marks:
            # Reject non-numeric values, including strings like "90"
            if isinstance(m, bool) or not isinstance(m, (int, float)):
                valid_marks = False
                break
            if m < 0 or m > 100:
                valid_marks = False
                break
            if m < 40:
                all_at_least_40 = False
            total += m

        if not valid_marks:
            errors.append(f"Record {index}: invalid marks")
            continue

        average = round(total / len(marks), 2)
        status = "PASS" if all_at_least_40 and average >= 50 else "FAIL"

        results.append({
            "name": name.strip(),
            "average": average,
```



```
        "status": status
    })

    return (results, errors)

# Edge cases handled:
# - Blank or space-only names are rejected, and if both fields are bad, only the name
  error is returned.
# - Empty marks lists, missing marks, string marks like "90", and values outside 0 to 100
  are rejected.
# - Valid records keep input order; boundary values 0 and 100 are accepted, and averages
  are rounded to 2 decimals.
```

HOW MARKS WERE AWARDED

3 marks: Defines process_results(records) with the exact required return shape (results, errors), preserves valid record order, and uses 0-based indexes in error messages. 4 marks: Correctly validates names and marks, including the edge cases of a blank/space-only name and an empty marks list. 3 marks: Correctly rejects non-numeric marks and marks outside 0 to 100 inclusive, including the edge cases of string numbers such as "90" and boundary values 0 and 100. 3 marks: Correctly computes average rounded to 2 decimals and status using both rules: every mark ≥ 40 and average ≥ 50 . 2 marks: Handles invalid records without crashing and reports only the invalid name when both name and marks are invalid.

The function structure is mostly correct, but there are multiple syntax errors, and it fails to return the expected results and errors.

QUESTION 31 · SCENARIO

8/10 marks

Your pod's Fault-Tolerant Student Result Processor reads a list of dictionaries and prints pass/fail results. A tester reports that the program sometimes marks students as FAIL even when their marks are valid. The only log lines are:

INFO Processing student: Asha



ERROR Could not process student record

INFO Processing student: Ravi

INFO Result: FAIL

A sample input file has records like:

```
{"name": "Asha", "marks": [80, 90, "absent"]}
```

```
{"name": "Ravi", "marks": [35, 36, 37]}
```

```
{"name": "Mina", "marks": []}
```

You are asked to fix this in a PR. What would you do first, and what should the PR include?

YOUR ANSWER

First, I would reproduce the sample data and inspect exception. the PR should validate that marks are non-empty list of numeric values treat invalid data like absent separately for a genuine FAIL, handle empty marks safely, log useful error context. I would also test for PASS, FAIL, non-numeric marks, empty marks and edge cases.

MODEL ANSWER

First, I would make the bug reproducible with the smallest possible input, using the provided three sample records exactly as they are. I would run the processor on just those records, confirm where Asha's record fails, and inspect the code path without changing multiple parts at once. That isolates whether the issue is in parsing, validation, averaging, or exception handling.

Then I would fix the data handling before the pass/fail logic. The processor should validate marks explicitly: reject or classify non-numeric values like "absent" as invalid, and handle an empty marks list separately. These cases should not be silently converted into FAIL, because FAIL implies a valid calculation that did not meet the threshold. Instead, they should produce a clear INVALID status with a reason.



I would also improve logging so each error includes useful context: student name, the invalid value or condition, and the exception type/message if an exception occurs. For example, log something like: "ERROR Student=Asha invalid mark='absent' ValueError: non-numeric mark". A generic "Could not process student record" is not enough for debugging.

The PR should also introduce a clear function contract. For example, a function can return PASS, FAIL, or INVALID along with a reason, and printing should be done separately from calculation. That keeps business logic, error handling, and output formatting from being mixed unpredictably.

Finally, I would add focused tests or manual test cases for: valid marks that pass, valid marks that fail, non-numeric marks like "absent", empty marks lists, and boundary cases around the pass mark. This would prove the fix and prevent regressions.

HOW MARKS WERE AWARDED

Award 10 marks: 2 marks for first making the bug reproducible with the given small sample data rather than changing many things at once; 2 marks for improving logs to include useful context such as student name, invalid value, exception type/message, while not logging only a generic error; 2 marks for validating marks before calculation, handling non-numeric values like 'absent' and empty lists explicitly instead of silently converting failures to FAIL; 2 marks for designing a clear function contract, for example returning a status such as PASS/FAIL/INVALID with a reason, instead of mixing printing, calculation, and error recovery unpredictably; 2 marks for adding focused tests or manual test cases covering valid marks, non-numeric marks, empty marks, and boundary pass/fail cases.

The student correctly identifies the need to reproduce the bug and validate input but lacks specifics on improving logging and defining a clear function contract.

QUESTION 32 · SCENARIO

6/10 marks

Euron wants a course FAQ assistant for Super30 students. It must answer questions such as setup issues, task deadlines, and Python assignment rules using only approved course notes. You are not allowed to fine-



tune a model. You can store the notes as chunks in Python data structures, retrieve relevant chunks, and send those chunks with the user question to the model. Design the retrieval, chunking, answer prompt, and evaluation plan. Also explain how you would reduce hallucinations and know whether the assistant is improving over time.

YOUR ANSWER

use RAG: split approved notes mainly by heading keep tables code blocks intact add small overlap and meta data such as page topic version. retrieve the most relevant chunks for each question and send only those with the question. Prompt the model to answer only from supplied context better to say I don't know from the approved notes when it is unable to find the evidence, first check and fix test set covering setup, deadlines, rules unsupported questions and outdated content measure retrieval accuracy. check all citation hallucination rate latency and improve it by reviewing and re run the same eval set

MODEL ANSWER

I would build the assistant as a retrieval-augmented system using only the approved course notes as its knowledge base, with no fine-tuning.

For chunking, I would split the notes by meaningful sections such as setup, task instructions, deadlines, Python assignment rules, submission policy, and FAQ items. I would keep tables, code blocks, and numbered steps intact so the meaning is not broken. Each chunk would store metadata such as source document name, heading, date/version, page number if available, and a unique chunk id. Where a section is long, I would use small overlap between adjacent chunks so that definitions or exceptions are not lost at boundaries.

For retrieval, I would first classify the user question into a topic like setup, deadline, or policy, then search only the chunks from the approved notes. I would rank chunks by similarity and return the top few most relevant ones, but only if they cross a confidence threshold. I would avoid sending unrelated chunks because that increases confusion and hallucination risk. If no chunk matches well, the system should say it could not find the answer in the approved notes and ask the student to rephrase or point to the relevant topic. The retrieved context should always include the source metadata and heading so the model can cite where the answer came from.



The answer prompt should force grounded responses: answer only from the supplied chunks, do not use outside knowledge, cite the source heading or chunk id, and if the evidence is missing or unclear, say you do not know from the notes. The prompt should also require concise student-friendly wording and prevent guessing. This reduces hallucinations because the model is explicitly constrained to the retrieved text and must refuse unsupported answers.

For evaluation, I would create a small test set of real student questions covering each note category, with expected answers and expected source chunks. I would measure exact answer accuracy, groundedness, and refusal quality for missing-information questions. I would also rerun the same tests after any note update to detect regressions.

To improve over time, I would log each question category, retrieved chunk ids, answer status, failed retrievals, and student feedback. That logging would show whether weak chunks need better splitting, whether the retriever is missing important sections, or whether the prompt needs tightening.

HOW MARKS WERE AWARDED

Chunking and metadata: 2 marks for chunking by meaningful sections such as setup, tasks, dates, and policies, keeping code/table content intact, adding metadata like source, heading, date/version, and using small overlap where needed. Retrieval design: 2 marks for describing how to select the most relevant chunks, avoid sending unrelated chunks, handle no-match cases, and include source metadata with the context. Answer prompt and hallucination control: 3 marks for instructing the model to answer only from supplied chunks, cite the source or heading, say it does not know when evidence is missing, and avoid using unsupported outside knowledge. Evaluation plan: 2 marks for proposing a small test set of real student questions with expected answers and sources, tracking accuracy, groundedness, refusal quality, and regression after note updates. Monitoring and improvement loop: 1 mark for logging question category, retrieved chunk ids, answer status, failures, and user feedback so weak chunks or prompts can be fixed over time.

The answer covers chunking and retrieval concepts but lacks detail in evaluation and monitoring plans, and some phrasing is unclear.



A validator has two checks for a combined loops-and-functions task. Out of 100 submissions, 70 pass the loops check, 60 pass the functions check, and 45 pass both checks. If one submission is chosen at random from the submissions that passed at least one of the two checks, what is the probability that it passed both?

45/100

45/70

9/17

CORRECT

$(70/100) * (60/100)$

QUESTION 34 · CODE

0/15 marks

Write a complete Python function named `organize_filenames(filenames)`.

Input: `filenames` is a list of strings representing file names, not full file contents.

Return a dictionary with exactly these keys: "images", "documents", "code", "others".

Each value must be a sorted list of unique original filenames that belong in that category.

Category rules are based on the file extension after the final dot, case-insensitive:

- images: jpg, jpeg, png, gif
- documents: pdf, docx, txt
- code: py, js, html, css



- others: no extension, unknown extension, hidden files like ".env" that have no extension before the final dot rule can apply

Important rules:

- Keep the original filename text in the output list; do not lowercase it.
- Ignore leading/trailing spaces only for deciding the extension, but store the stripped filename in the output.
- If the same stripped filename appears more than once, include it once.
- A filename ending with a dot, such as "report.", has no valid extension and goes to others.
- Sort each category list alphabetically using Python's normal string sorting.

Example:

Input:

```
[" photo.JPG ", "main.py", "README", ".env", "report.", "notes.TXT", "photo.JPG", "archive.tar.gz", "index.HTML"]
```

Return:

```
{  
  "images": ["photo.JPG"],  
  "documents": ["notes.TXT"],  
  "code": ["index.HTML", "main.py"],  
  "others": [".env", "README", "archive.tar.gz", "report."] }  
}
```

YOUR ANSWER

– no answer –



MODEL ANSWER

```
def organize_filenames(filenames):
    # Required categories
    categories = {
        "images": [],
        "documents": [],
        "code": [],
        "others": []
    }

    # Extension to category mapping (case-insensitive)
    ext_map = {
        "jpg": "images", "jpeg": "images", "png": "images", "gif": "images",
        "pdf": "documents", "docx": "documents", "txt": "documents",
        "py": "code", "js": "code", "html": "code", "css": "code"
    }

    seen = set() # track stripped filenames to remove duplicates

    for name in filenames:
        stripped = name.strip()
        if stripped in seen:
            continue
        seen.add(stripped)

        # Determine extension from the final dot only
        # Hidden files like ".env" and names ending with a dot go to others
        ext = None
        if stripped and not stripped.endswith('.'):
            dot_index = stripped.rfind('.')
            if dot_index > 0 and dot_index < len(stripped) - 1:
                ext = stripped[dot_index + 1:].lower()

        if ext in ext_map:
            categories[ext_map[ext]].append(stripped)
        else:
            categories["others"].append(stripped)

    # Sort each list using normal Python string sorting
    for key in categories:
        categories[key].sort()

    return categories
```



Edge cases handled:

1) Leading/trailing spaces are stripped before duplicate checking, classification, and output.

2) Hidden files like ".env", no-extension names like "README", and trailing-dot names like "report." all go to others.

3) Multi-dot names such as "archive.tar.gz" use only the final extension ("gz"), and unknown extensions safely go to others.

HOW MARKS WERE AWARDED

3 marks: Returns a dictionary with exactly the four required keys and sorted list values, even when a category is empty. 3

marks: Correctly strips filenames for processing and output while preserving original casing, and removes duplicate

stripped filenames. 3 marks: Correctly extracts the extension after the final dot and applies case-insensitive category

matching. 3 marks: Correctly handles edge cases including hidden files like ".env" and filenames ending with a dot such as

"report." as others. 3 marks: Correctly handles no-extension and unknown-extension filenames, including multi-dot names

such as "archive.tar.gz", without crashing.

No answer provided.

